

Malicious Traffic Detection in CICIDS-2017 Using Machine Learning

Matheus Fraga Cardoso, Vinícius Guerra de Mello

¹Programa de Pós-Graduação em Computação (PPGC) – Universidade Federal do Rio Grande do Sul
Porto Alegre – RS – Brasil

Abstract. *This work presents a reproducible study of binary malicious traffic detection on the CICIDS-2017 dataset. We applied a machine learning pipeline with exploratory data analysis, documented preprocessing, stratified holdout split, spot-checking of five classifiers, repeated stratified cross-validation (10 folds $\times$ 3 repetitions), hyperparameter tuning of the two best models, and interpretability via Gini importance and SHAP. The optimized random forest achieved an F1-score of 0.9980 for the Attack class on the test holdout, with a train–test gap of 0.0005, indicating good generalization. Tree-based models and ensembles outperformed logistic regression and Gaussian naive Bayes; tuning yielded marginal gains (+0.0008 and +0.0004 in F1). Gini and SHAP analyses converged on Destination Port and backward flow statistics as the most discriminative attributes. We prioritized reproducibility: code, models, intermediate artifacts, and the preprocessed dataset are publicly available, enabling full reproduction with artifact caching. Furthermore, the pipeline can be reproduced on any machine with the same dependencies using a Docker container.*

Resumo. *Este trabalho apresenta um estudo reprodutível de detecção binária de tráfego malicioso no dataset CICIDS-2017. Aplicamos um pipeline de aprendizado de máquina com análise exploratória, pré-processamento documentado, divisão estratificada holdout, spot-checking de cinco classificadores, validação cruzada estratificada repetida (10 folds $\times$ 3 repetições), otimização de hiperparâmetros dos dois melhores modelos e interpretabilidade por importância de Gini e SHAP. O modelo random forest otimizado alcançou F1 de 0,9980 para a classe Attack no holdout de teste, com gap treino–teste de 0,0005, indicando boa generalização. Modelos baseados em árvores e ensembles superaram regressão logística e Naive Bayes gaussiano; o tuning trouxe ganhos marginais (+0,0008 e +0,0004 em F1). As análises de Gini e SHAP convergiram para Destination Port e estatísticas de fluxo backward como atributos mais discriminativos. Priorizamos a reprodutibilidade: código, modelos, artefatos intermediários e dataset pré-processado estão publicamente disponíveis, permitindo reprodução completa com cache de artefatos. Além disso, o pipeline pode ser reproduzido em qualquer máquina com as mesmas dependências, com o uso de um container Docker.*

1. Introdução

A detecção de tráfego malicioso é um problema central em cibersegurança. Segundo o Verizon Data Breach Investigations Report (DBIR) de 2026 [Verizon Business 2026], os

maiores ataques distribuídos de negação de serviço (DDoS) apresentaram crescimento de 198% no volume de bits por segundo; a exploração de vulnerabilidades tornou-se o principal vetor inicial de invasão, representando 31% das violações registradas e crescimento de 55% em relação ao período anterior; e o uso de inteligência artificial por agentes maliciosos acelera a descoberta de falhas, automatiza a exploração de vulnerabilidades e potencializa etapas dos ataques cibernéticos, aumentando a complexidade dos mecanismos tradicionais de defesa.

Nesse cenário, os sistemas de detecção de intrusão (IDS) baseados em assinatura dependem de padrões conhecidos e bem definidos, mas apresentam limitações claras frente a ataques novos ou ofuscados, o que motiva abordagens estatísticas e de aprendizado de máquina (AM) capazes de generalizar para padrões não vistos. Além disso, abordagens baseadas em AM enfrentam desafios adicionais, como a escassez de *datasets* de referência publicamente disponíveis e o envelhecimento rápido desses *benchmarks* à medida que as tecnologias de rede evoluem [He et al. 2023].

Nesse contexto, este trabalho treina modelos de aprendizado de máquina para detecção binária de tráfego malicioso, a fim de obter classificadores capazes de distinguir tráfego legítimo e malicioso e de generalizar para ataques novos ou ofuscados, superando as limitações das abordagens por assinatura e mitigando os efeitos do envelhecimento de *datasets*. Para isso, utiliza o *dataset* CICIDS-2017 [Sharafaldin et al. 2018], amplamente adotado como *benchmark* de detecção de intrusão, com 14 tipos de ataques rotulados e mais de 2 milhões de amostras de fluxos de rede. Este trabalho também explora a interpretabilidade dos modelos treinados e a reprodutibilidade dos experimentos propostos.

As principais contribuições deste trabalho são: (i) comparação sistemática de cinco classificadores para detecção binária no CICIDS-2017; (ii) análise de interpretabilidade do melhor modelo (SHAP, importância de atributos); (iii) *pipeline* reproduzível (código/Docker).

O restante deste artigo está organizado da seguinte forma. A Seção 2 revisa trabalhos relacionados à detecção de intrusão com aprendizado de máquina. A Seção 3 descreve a metodologia, incluindo o *dataset*, o pré-processamento, o *pipeline* de treinamento, as métricas de avaliação, a interpretabilidade dos modelos, a reprodutibilidade dos experimentos e o uso de inteligência artificial generativa (Seção 3.7). A Seção 4 apresenta os experimentos e os resultados obtidos. A Seção 5 discute os achados e limitações do estudo. Por fim, a Seção 6 apresenta as conclusões e perspectivas de trabalhos futuros.

2. Trabalhos relacionados

A literatura sobre detecção de tráfego malicioso em aprendizado de máquina (AM) consolidou o uso de *benchmarks* públicos de tráfego rotulado para comparar modelos e relatórios de desempenho. Entre esses conjuntos, o CICIDS-2017 [Sharafaldin et al. 2018] tornou-se referência recorrente por reproduzir fluxos benignos e maliciosos com dezenas de atributos derivados de fluxos de rede. Nesta seção, revisamos três trabalhos representativos: dois estudos empíricos no CICIDS-2017 e um *survey* sobre AM adversarial em NIDS.

Jairu e Mailewa [Jairu and Mailewa 2022] propõem uma abordagem supervisionada para descoberta de anomalias de rede no CICIDS-2017. O estudo enfatiza a insu-

ficiência de IDS baseados apenas em assinaturas e avalia múltiplos modelos supervisionados sobre características de fluxo, com foco em identificar padrões de tráfego malicioso frente a tráfego legítimo. Os autores discutem o papel do pré-processamento e da seleção de atributos para elevar a acurácia na detecção de ataques conhecidos no *dataset*, posicionando o AM como alternativa prática à detecção puramente baseada em regras.

Maseer *et al.* [Maseer et al. 2021] conduzem um *benchmark* de algoritmos de AM para IDS baseados em anomalias no CICIDS2017. O artigo compara dez técnicas supervisionadas e não supervisionadas—incluindo árvores de decisão, *k*-NN, Naive Bayes, floresta aleatória, SVM e redes neurais—em dezenas de configurações de modelos, reportando acurácia, precisão, *recall*, F1 e tempo de treinamento. Os resultados indicam que modelos como *k*-NN, árvore de decisão e Naive Bayes obtêm desempenho competitivo em classes de ataques específicas, mas o desbalanceamento e a diversidade de ataques no CICIDS2017 exigem critérios de avaliação cuidadosos.

Em perspectiva mais ampla, He *et al.* [He et al. 2023] apresentam um *survey* sobre aprendizado de máquina adversarial aplicado a sistemas de detecção de intrusão em rede (NIDS). Os autores organizam taxonomias de NIDS baseados em aprendizado profundo, formulam ataques adversariais voltados a evasão de modelos e revisam mecanismos de defesa. O levantamento destaca desafios estruturais da área—como escassez de *datasets* públicos confiáveis, obsolescência rápida de *benchmarks* e limitações de interpretabilidade em modelos de alto desempenho—que motivam avaliações reproduzíveis e análises explicáveis, e não apenas ganhos marginais de acurácia.

Posicionamento deste trabalho. Os estudos de Jairu e Mailewa [Jairu and Mailewa 2022] e de Maseer *et al.* [Maseer et al. 2021] concentram-se principalmente no desempenho preditivo de classificadores no CICIDS-2017, em geral em cenários multiclasse. O *survey* de He *et al.* [He et al. 2023] sistematiza riscos e lacunas de pesquisa, mas não propõe um *pipeline* experimental único com foco em interpretabilidade. Este trabalho diferencia-se por: (i) adotar detecção binária (tráfego legítimo *versus* malicioso) com cinco classificadores clássicos; (ii) analisar interpretabilidade do melhor modelo (por exemplo, SHAP e medidas de importância de atributos); e (iii) documentar a reprodutibilidade dos experimentos (código e ambiente containerizado). Assim, complementamos *benchmarks* existentes no CICIDS-2017 com uma visão orientada à explicabilidade e à replicação, em linha com as recomendações de pesquisa apontadas por He *et al.* [He et al. 2023].

3. Metodologia

3.1. Conjunto de dados

O *dataset* utilizado é o CICIDS-2017 [Sharafaldin et al. 2018], produzido pelo Canadian Institute for Cybersecurity. O conjunto compreende 14 tipos de ataques, em um cenário de classificação multiclasse nos rótulos originais; neste trabalho, os fluxos são agregados para detecção binária (tráfego legítimo *versus* malicioso).

A distribuição agregada dos rótulos é severamente desbalanceada: 80,3% de fluxos BENIGN e 19,7% ATTACK após unificar os tipos de ataque (Figura 1). Entre as classes de ataque individuais, a disparidade é ainda maior; categorias raras, como Heartbleed (7 amostras) e Web_Attack_Sql_Injection (21 amostras), tornam inviável um treinamento multiclasse estável no escopo deste estudo (Figura 2).

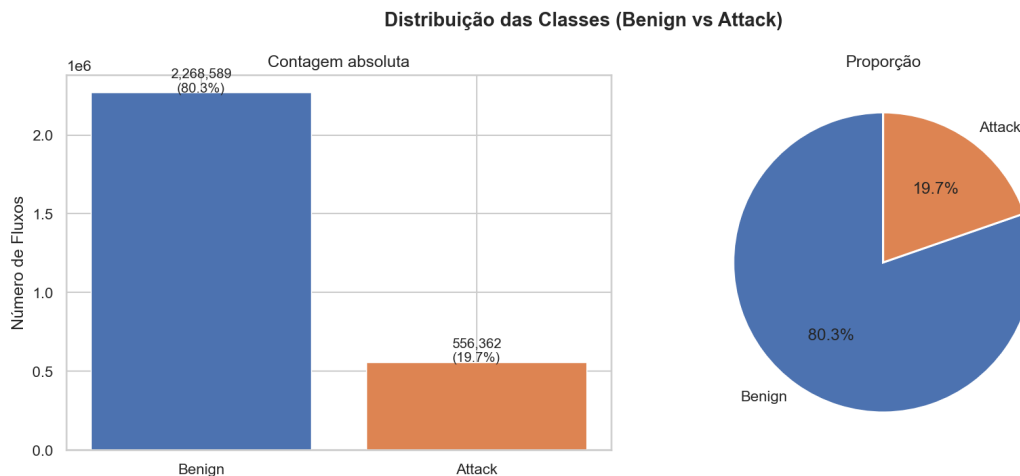


Figura 1. Distribuição agregada BENIGN *versus* ATTACK no CICIDS-2017.

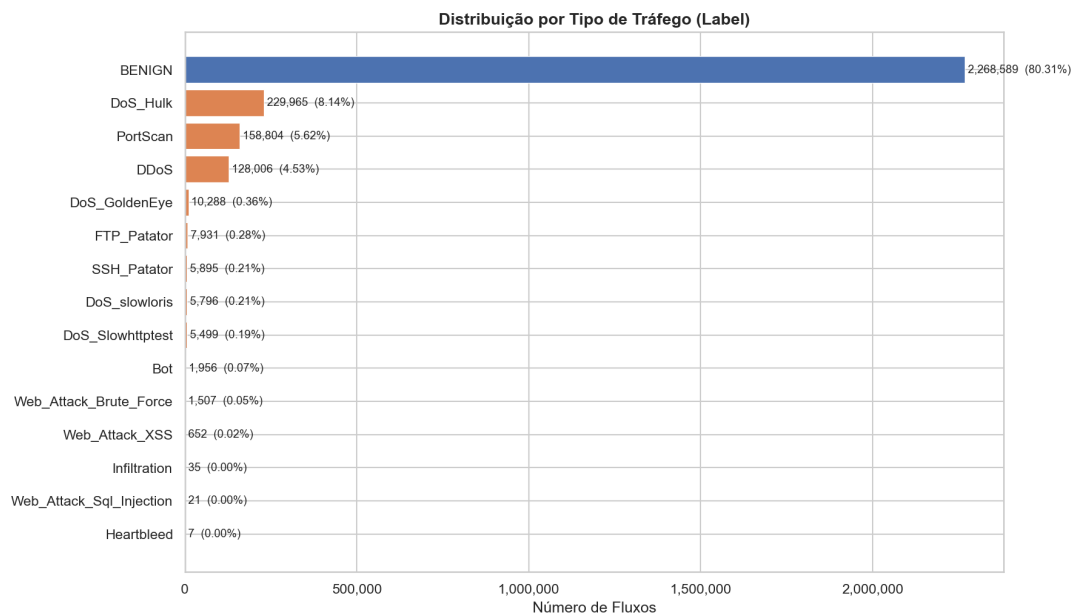


Figura 2. Distribuição por tipo de ataque no CICIDS-2017.

3.1.1. Formulação binária

Dada a dificuldade de aprender classes com menos de 40 amostras no prazo do projeto, agrupamos todos os ataques na classe ATTACK e definimos a tarefa como classificação binária (BENIGN *versus* ATTACK). Reconhecemos que essa escolha reduz a granularidade por tipo de ataque; essa limitação é discutida na Seção 5.

3.2. Dados e pré-processamento

Unificamos os oito arquivos CSV do CICIDS-2017 e aplicamos um *pipeline* de limpeza e redução de dimensionalidade, guiado por análise exploratória (EDA), antes do treinamento dos classificadores.

Os problemas tratados foram:

- **Duplicatas:** 255.234 linhas repetidas (9,03% do total), removidas com base nas

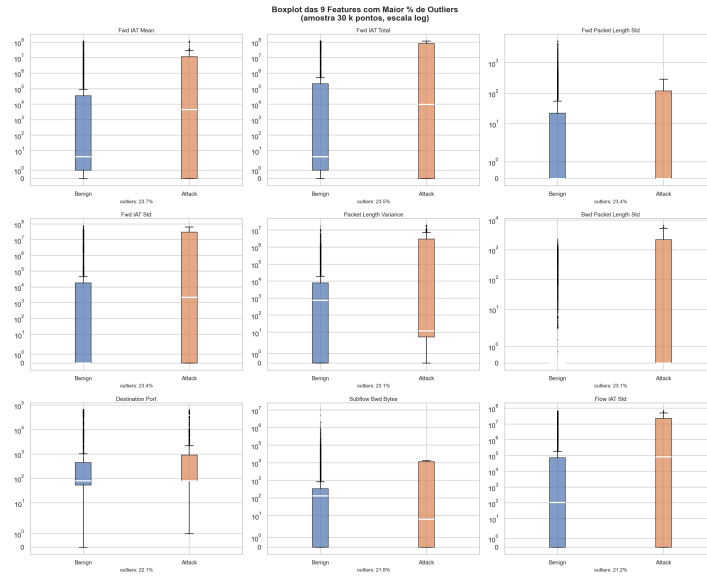


Figura 3. Boxplots dos atributos com maior incidência de outliers (IQR).

colunas de fluxo (excluindo metadados de origem do arquivo).

- **Valores infinitos:** nas colunas Flow Bytes/s e Flow Packets/s, gerados por divisão por zero; substituídos por NaN e descartados na limpeza subsequente.
- **Variância zero:** oito atributos constantes em todo o conjunto (*_Avg_Bulk_*, Bwd PSH Flags, Bwd URG Flags), eliminados por não contribuírem para a discriminação.
- **Coluna redundante:** Fwd Header Length.1, idêntica a Fwd Header Length, removida.
- **Alta correlação:** remoção iterativa de atributos com $|r| \geq 0,95$ (Pearson), resultando em 23 atributos redundantes adicionais descartados.
- **Outliers:** tratamento pelo método do intervalo interquartil (IQR), com limitação suave (*clipping*) nos extremos das variáveis numéricas, conforme a distribuição observada na EDA (Figura 3).

Após a agregação binária dos rótulos, a matriz final contém 47 atributos numéricos e 2.824.951 fluxos.

3.3. Pipeline de treinamento

O treinamento foi implementado em Python 3.11 com *scikit-learn*, em cinco etapas, com `random_state=42`, `class_weight=balanced` (sem SMOTE) e métricas sensíveis ao desbalanceamento (Seção 3.4).

Algoritmos avaliados. Comparamos cinco classificadores clássicos, escolhidos por interpretabilidade, custo computacional e uso recorrente em *benchmarks* de IDS: Regressão Logística (RL), Random Forest (RF), Árvore de Decisão (AD), Hist Gradient Boosting (HGB) e Naive Bayes Gaussiano (NB). A RL é treinada em um Pipeline com `StandardScaler` ajustado dentro de cada *fold* de validação, prevenindo *data leakage* do conjunto de validação no escalonamento; árvores e *ensemble* baseados em árvores (RF, AD, HGB) operam diretamente sobre os atributos originais.

Divisão dos dados. Aplicamos *holdout* estratificado 80/20 sobre a matriz final de fluxos, preservando a proporção BENIGN/ATTACK em treino e teste. O conjunto de teste

permaneceu isolado de todas as etapas subsequentes—validação cruzada e otimização de hiperparâmetros—e foi utilizado apenas na avaliação final reportada.

Experimentos.

1. **Spot-checking:** treino dos cinco modelos com hiperparâmetros padrão da biblioteca (por exemplo, RL com `max_iter=1000`, RF com `n_estimators=100`) sobre o conjunto de treino e avaliação no conjunto de teste, para triagem inicial.
2. **Validação cruzada:** os mesmos modelos foram reavaliados por *Repeated Stratified K-Fold* com $k = 10$ folds e três repetições (30 avaliações por modelo), exclusivamente no conjunto de treino, reportando média e desvio padrão de F1, precisão, *recall* e ROC-AUC (classe positiva ATTACK).
3. **Otimização de hiperparâmetros:** para os dois modelos mais promissores nas etapas anteriores (RF e HGB), *RandomizedSearchCV* com 20 combinações amostradas e *StratifiedKFold* interno com cinco folds; por viabilidade computacional, a busca usou uma amostra estratificada de 20% do treino, e o melhor modelo foi retreinado no treino completo.
4. **Avaliação final:** modelo otimizado avaliado no *holdout* de teste reservado.

3.4. Métricas e avaliação

Avaliamos os modelos com quatro métricas calculadas no *scikit-learn*, sempre com classe positiva Attack (tráfego malicioso): precisão (fração de alertas corretos entre os previstos como ataque), *recall* (fração de ataques reais detectados), F1 (média harmônica entre precisão e *recall*) e ROC-AUC (capacidade de separar as classes ao variar o limiar de decisão). Também reportamos acurácia (*accuracy*) como referência, mas não a usamos para ranquear modelos.

Adotamos o F1 da classe Attack como critério principal de comparação e seleção dos modelos. Com cerca de 80% de fluxos Benign e 20% Attack, um classificador que prevê sempre tráfego legítimo obtém acurácia elevada, porém F1 nulo para ataques—inadequado para detecção de intrusão, em que falhar em detectar Attack é o erro mais grave. O F1 equilibra precisão e *recall*: alta precisão reduz falsos alarmes; alto *recall* aumenta a cobertura de ataques reais.

No *spot-checking*, calculamos as cinco métricas no conjunto de teste reservado, geramos matriz de confusão e *report* de classificação por modelo, e ordenamos os resultados pelo F1 de Attack. Na validação cruzada (*Repeated Stratified K-Fold*, 30 avaliações por algoritmo), registramos média e desvio padrão de F1, precisão, *recall* e ROC-AUC apenas no conjunto de treino. Após a otimização de hiperparâmetros (RF e HGB), a avaliação final no *holdout* de teste repetiu o mesmo conjunto de métricas para estimar o desempenho de generalização.

3.5. Interpretabilidade

Além do desempenho preditivo, analisamos *por que* o classificador separa tráfego legítimo e malicioso. Tomamos como referência o **Random Forest otimizado**, retreinado no conjunto de treino completo após *RandomizedSearchCV*: no *holdout*, obteve o maior F1 da classe Attack entre os modelos ajustados (RF e HGB), além de ser naturalmente interpretável por árvores. Todas as análises abaixo usam as mesmas 47 features produzidas na Seção 3.2 (espaço de atributos de treino, teste e SHAP).

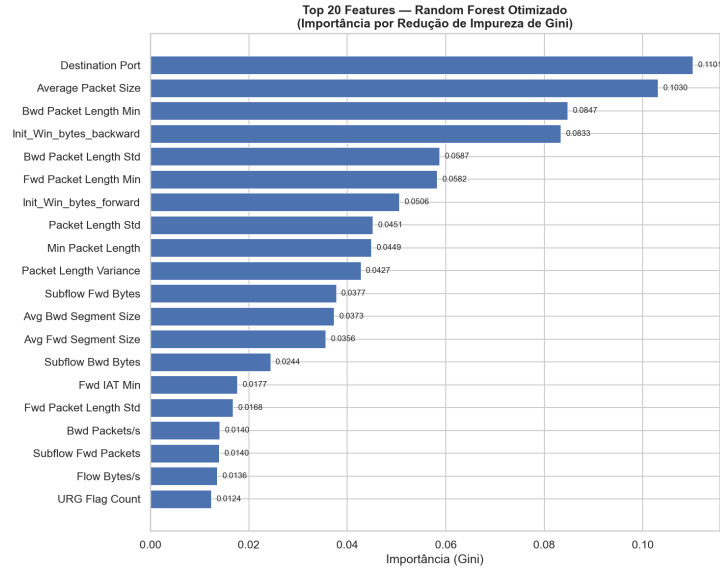


Figura 4. Top-20 atributos por importância de Gini no `rf_opt`.

3.5.1. Importância global (Gini)

Ranqueamos as 47 features pelas importâncias de atributos do `rf_opt` ajustado em X_{train} ; na subseção seguinte, complementamos com SHAP. A Figura 4 mostra os 20 atributos mais relevantes.

3.5.2. Explicações locais (SHAP)

O protocolo SHAP é fixo e reproduzível: o modelo é o `rf_opt` retreinado em X_{train} completo (sem novo ajuste de hiperparâmetros); as contribuições são calculadas com `TreeExplainer` [Lundberg and Lee 2017] sobre uma amostra aleatória de **5.000** fluxos de X_{test} (`random_state=42`), para a classe `Attack` (valores SHAP positivos favorecem `Attack`; negativos, `Benign`). Trata-se de explicação **pós-hoc**: não há retreino do classificador nem seleção ou ajuste de atributos com base no conjunto de teste—o SHAP apenas quantifica, para o modelo já fixo, quanto cada uma das 47 features altera a predição em cada fluxo amostrado.

A Figura 5 resume o efeito por feature: `Destination Port` e features *backward* concentram contribuições positivas para `Attack`, alinhadas ao Gini. Para as cinco variáveis de maior $|SHAP|$ médio, a Figura 6 traz *dependence plots* (efeito marginal e não linearidades).

Gini e SHAP concordam nas mesmas features principais: `Destination Port`, fluxo *backward* e tamanhos de pacote.

Limitações. A importância de Gini não desagrega efeitos entre atributos correlacionados; o SHAP foi avaliado em 5.000 fluxos de teste, não no *holdout* inteiro; `TreeExplainer` aplica-se a modelos baseados em árvore (aqui, RF) e não se estende, sem adaptação, a outros classificadores.

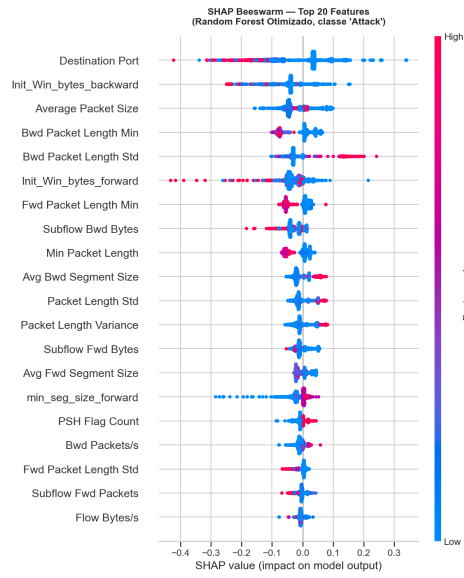


Figura 5. Resumo SHAP (*beeswarm*), classe *Attack* ($n = 5.000$, teste).

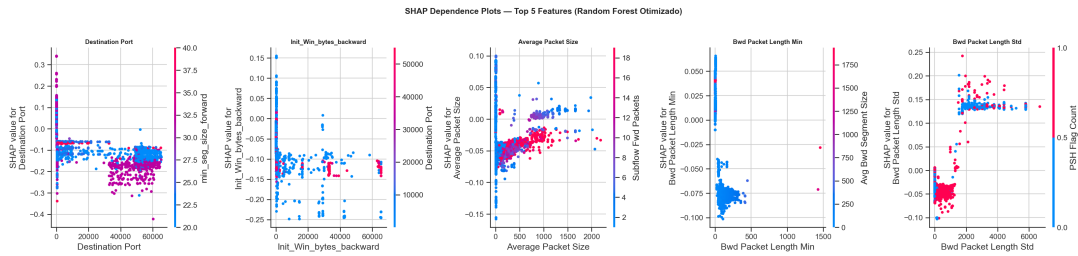


Figura 6. Dependence plots SHAP: top-5 por $|SHAP|$ médio (classe *Attack*).

3.6. Reprodutibilidade

O *pipeline* está em projeto/códigos/notebook_completo.ipynb no repositório <https://github.com/vinigm/projeto-ddos> (commit 2b0b79d), com caminhos relativos a ../archive/ (figuras, raw_limpo.parquet) e ../../artifacts/ (cache). O *parquet* pré-processado (47 atributos; Seção 3.2) é baixado da release v1.0 se não existir localmente. Todas as etapas estocásticas usam `random_state=42`; dependências fixadas em `requirements.txt` (Python ≥ 3.10 ; `scikit-learn==1.8.0`, `shap==0.51.0`, entre outras). Com `RECOMPUTE=False`, modelos, escores de CV, hiperparâmetros e valores SHAP são lidos de `artifacts/` quando disponíveis; `RECOMPUTE=True` refaz o treino completo. Para replicar resultados e figuras, use *Run All Below* a partir da seção “Pós EDA” do notebook (com `RECOMPUTE=False` quando o cache existir); o guia `DOCKER.md` descreve a execução com `docker compose up --build` [Boettiger 2015]. Artefatos volumosos podem não estar versionados; a primeira replicação pode exigir gerá-los localmente.

3.7. Uso de inteligência artificial

Para o desenvolvimento do trabalho, declaramos que utilizamos ferramentas de inteligência artificial, mais especificamente o *Cursor* (assinatura **Pro**, período de elaboração do trabalho) como apoio à redação e à engenharia do repositório. Os modelos empre-

gados nas sessões, conforme a tarefa, incluíram *Composer 2.5 Fast*, *Composer 2 Fast*, Claude 4.6 Sonnet, Claude Opus 4.7, GPT-5.3 Codex e GPT-5.5 Medium.

O fluxo adotado no *Cursor* foi:

1. **Ask Mode:** discussão do objetivo e rascunho de *prompts* e de plano de ação (sem alterar arquivos automaticamente).
2. **Plan Mode:** plano de implementação ou de revisão textual, com engenharia de *prompt* sobre o código e o artigo.
3. **Agent Mode:** execução do plano aprovado (edições em $\text{\LaTeX}$, notebook, documentação), com revisão dos autores.

A IA **não** rodou os experimentos de aprendizado de máquina nem definiu métricas ou conclusões. A ferramenta apoiou revisão de texto, formatação do artigo e tarefas repetitivas de código.

4. Experimentos e resultados

Esta seção reporta os resultados do *pipeline* descrito na Seção 3, na ordem em que os experimentos foram conduzidos no `notebook_completo.ipynb`.

4.1. Spot-checking inicial

A Tabela 1 apresenta o desempenho dos cinco algoritmos no *holdout* de teste. Random Forest e Hist Gradient Boosting destacam-se como os candidatos mais promissores, com F1 da classe *Attack* acima de 0,997. Naive Bayes apresenta o pior desempenho (F1 = 0,5744).

4.2. Validação cruzada

A Tabela 2 reporta os resultados do *Repeated Stratified K-Fold* (30 avaliações) sobre o conjunto de treino. Random Forest, Árvore de Decisão e Hist Gradient Boosting apresentam desempenho praticamente idêntico, com F1 acima de 0,996 e desvios padrão baixos ($\leq 0,0004$), indicando alta estabilidade entre diferentes partições dos dados.

Tabela 1. Spot-checking inicial – desempenho dos cinco algoritmos no conjunto de teste (*holdout*). Métricas reportadas para a classe positiva (*Attack*).

Modelo	Acurácia	Precisão	<i>Recall</i>	F1- <i>Attack</i>	ROC-AUC
Regressão Logística	0,9405	0,7890	0,9528	0,8632	0,9816
Random Forest	0,9989	0,9973	0,9972	0,9972	0,9998
Árvore de Decisão	0,9989	0,9971	0,9971	0,9971	0,9982
Hist Gradient Boosting	0,9990	0,9955	0,9996	0,9975	0,9999
Naive Bayes	0,8645	0,7526	0,4645	0,5744	0,7268

Tabela 2. Resultados da validação cruzada *Repeated Stratified K-Fold* (10 *folds* $\times$ 3 repetições). Valores: média $\pm$ desvio padrão sobre as 30 avaliações.

Modelo	F1- <i>Attack</i>	Precisão	<i>Recall</i>	ROC-AUC
Regressão Logística	0,8640 $\pm$ 0,0012	0,7905 $\pm$ 0,0019	0,9527 $\pm$ 0,0011	0,9818 $\pm$ 0,0003
Random Forest	0,9973 $\pm$ 0,0001	0,9972 $\pm$ 0,0002	0,9974 $\pm$ 0,0002	0,9998 $\pm$ 0,0000
Árvore de Decisão	0,9971 $\pm$ 0,0001	0,9967 $\pm$ 0,0003	0,9975 $\pm$ 0,0003	0,9984 $\pm$ 0,0001
Hist Gradient Boosting	0,9969 $\pm$ 0,0004	0,9942 $\pm$ 0,0008	0,9996 $\pm$ 0,0001	0,9999 $\pm$ 0,0000
Naive Bayes	0,5753 $\pm$ 0,0024	0,7531 $\pm$ 0,0029	0,4654 $\pm$ 0,0022	0,7265 $\pm$ 0,0013

4.3. Otimização de hiperparâmetros

A Tabela 3 reporta os hiperparâmetros selecionados pelo *RandomizedSearchCV*. Para o RF, a busca encontrou `max_depth=30`, `min_samples_leaf=5`, `max_features=0,3` e `n_estimators=100`, com F1 de validação cruzada de 0,9974. Para o HGB, os parâmetros selecionados foram `max_iter=500`, `learning_rate=0,1`, `max_depth=None`, `min_samples_leaf=20` e `l2_regularization=0,1`, com F1 de 0,9972. A Tabela 4 compara o desempenho dos modelos *baseline* e otimizados no conjunto de teste final. Os ganhos com a otimização foram marginais (RF: +0,0008 em F1; HGB: +0,0004), sugerindo que o desempenho *baseline* já se aproximava de um teto natural para esta formulação binária do problema.

Tabela 3. Hiperparâmetros selecionados pelo *RandomizedSearchCV* (20 amostragens, 5-fold CV em 20% do treino).

Modelo	Hiperparâmetro	Valor selecionado
Random Forest	<code>n_estimators</code>	100
	<code>max_depth</code>	30
	<code>min_samples_leaf</code>	5
	<code>max_features</code>	0,3
Hist Gradient Boosting	<code>max_iter</code>	500
	<code>learning_rate</code>	0,1
	<code>max_depth</code>	<i>None</i>
	<code>min_samples_leaf</code>	20
	<code>l2_regularization</code>	0,1

Tabela 4. Comparação *baseline* versus otimizado no conjunto de teste (*holdout*).

Modelo	Precisão	<i>Recall</i>	F1-Attack	ROC-AUC
RF <i>baseline</i>	0,9973	0,9972	0,9972	0,9998
RF otimizado	0,9968	0,9992	0,9980	0,9999
HGB <i>baseline</i>	0,9955	0,9996	0,9975	0,9999
HGB otimizado	0,9961	0,9997	0,9979	1,0000

4.4. Validação contra overfitting

Para verificar a ausência de overfitting, comparamos o F1 nos conjuntos de treino e teste para todos os modelos avaliados (Tabela 5). O *gap* entre F1 de treino e teste é menor que 0,25% em todos os casos, e o modelo final (RF otimizado) apresenta *gap* de apenas +0,0005 — evidência forte de que o modelo generaliza bem.

Tabela 5. Comparação F1 treino *versus* teste como diagnóstico de overfitting.

Modelo	F1 treino	F1 teste	Gap
Regressão Logística	0,8643	0,8632	+0,0010
Random Forest <i>baseline</i>	0,9994	0,9972	+0,0022
Árvore de Decisão	0,9994	0,9971	+0,0023
HGB <i>baseline</i>	0,9975	0,9975	+0,0000
Naive Bayes	0,5753	0,5744	+0,0009
RF otimizado	0,9985	0,9980	+0,0005
HGB otimizado	0,9980	0,9979	+0,0000

5. Discussão

Com os experimentos propostos, foi possível verificar que os modelos baseados em árvores e *ensembles* (RF, AD e HGB) atingem F1 da classe `Attack` acima de 0,996 tanto no *holdout* (Tabela 1) quanto na validação cruzada repetida (Tabela 2), com desvios padrão de até 0,0004. A Regressão Logística permanece competitiva para um modelo linear ($F1 \approx 0,86$), em contrapartida o Naive Bayes obteve um desempenho pior ($F1 \approx 0,57$), coerente com a assimetria e a escala das features de fluxo após o pré-processamento.

O uso de `class_weight=balanced`, sem SMOTE, mostrou-se suficiente para elevar *recall* de `Attack` nos modelos fortes, mas não resolve o problema estrutural das classes raras na visão multiclasse original (Seção 3).

5.1. Desempenho e tuning

A otimização de hiperparâmetros (Seção 4.3) elevou o F1 do RF em apenas +0,0008 e o do HGB em +0,0004 frente aos *baselines*, enquanto o diagnóstico treino–teste (Tabela 5) manteve *gaps* abaixo de 0,25% em todos os modelos. Esse padrão sugere que, após a limpeza guiada por EDA, a redução de dimensionalidade para 47 atributos e a agregação binária dos rótulos, o problema fica próximo de um teto informativo em torno de $F1 \approx 0,997$ —ajustes finos alteram pouco a fronteira de decisão já capturada por árvores profundas e *ensembles*. Em contraste com *benchmarks* que enfatizam apenas acurácia global [Maseer et al. 2021], o critério F1 de `Attack` revelou que métricas insensíveis ao desbalanceamento mascarariam falhas graves de detecção (um classificador que prevê sempre `Benign` teria acurácia elevada e F1 nulo para ataques).

5.2. Modelo recomendado

Adotamos o **Random Forest otimizado** (`rf_opt`) como modelo final, com base em três critérios:

1. **Desempenho no *holdout*:** F1 de `Attack` = 0,9980, superior ao HGB otimizado (0,9979; Tabela 4).
2. **Estabilidade na CV:** desvio padrão de 0,0001 no F1, contra 0,0004 do HGB (Tabela 2)—quatro vezes mais estável entre *folds*.
3. **Interpretabilidade:** `TreeExplainer` fornece explicações para florestas aleatórias (Seção 3.5), sendo esse um pilar extremamente importante para o trabalho proposto.

Na validação cruzada, RF, AD e HGB diferem em menos de 0,0004 pontos de F1 médio; em termos práticos, os três são equivalentes para ranqueamento neste *benchmark*. A escolha do RF é, portanto, um compromisso operacional entre desempenho marginal, estabilidade e auditabilidade. Em cenários com restrição severa de memória ou latência de inferência, o HGB otimizado permanece alternativa plausível, dado desempenho quase idêntico e *gap* treino–teste nulo na Tabela 5.

5.3. Interpretabilidade e implicações operacionais

Gini e SHAP (Seção 3.5) apontam as mesmas features centrais—*Destination Port*, janelas TCP iniciais e estatísticas de fluxo *backward*—coerentes com padrões esperados em IDS. O *ranking* de features ajuda a priorizar alertas e a verificar se o modelo captou tráfego de rede plausível; não dispensa inspeção de pacotes, inteligência de ameaças nem validação fora do *testbed*.

5.4. Limitações

Apesar da metodologia proposta, permanecem algumas limitações relevantes:

1. **Holdout único:** reservamos 20% dos fluxos para teste e medimos o modelo final neles uma única vez. Comparação de algoritmos, validação cruzada e busca de hiperparâmetros ocorreram só no treino—o teste não participou dessas etapas. Repetir todo o *pipeline* com validação cruzada aninhada (em cada *fold*, novo treino e novo ajuste) daria uma estimativa mais estável do desempenho final, mas o custo com ~2,3 milhões de fluxos de treino tornou essa opção inviável.
2. **Busca de hiperparâmetros em 20% do treino:** para reduzir tempo, o *RandomizedSearchCV* testou 20 combinações de hiperparâmetros em apenas 20% do conjunto de treino (amostra estratificada). Os melhores valores escolhidos podem não ser os mesmos que apareceriam se a busca rodasse nos 100% do treino; depois da busca, o modelo vencedor foi retreinado com todo o treino, mas já com hiperparâmetros fixados pela subamostra.
3. **Formulação binária:** unificar 14 tipos de ataque em ATTACK sacrifica granularidade para resposta a incidentes (DDoS, exfiltração, varredura, etc.).
4. **Escopo do dataset:** resultados limitam-se ao CICIDS-2017, gerado em *testbed* controlado. Seria de grande valor testar em outros *benchmarks*.
5. **Interpretabilidade amostrada:** SHAP foi calculado em 5.000 fluxos de teste, não no *holdout* completo.

6. Conclusão

Este trabalho apresentou um estudo, com foco em reprodutibilidade, sobre detecção de tráfego malicioso, comparando cinco modelos clássicos sobre um *pipeline* documentado de EDA, pré-processamento, *spot-checking*, validação cruzada repetida estratificada, otimização de hiperparâmetros e interpretabilidade (Gini e SHAP). As principais contribuições são: (i) evidência empírica de que *ensembles* de árvores superam amplamente modelos lineares e probabilísticos simples nesta formulação binária, após limpeza e seleção de atributos; (ii) análise explicativa convergente do `rf_opt`; e (iii) replicação facilitada por código, dependências fixadas, e de artefatos no repositório público (Seção 3.6). Além disso, o *pipeline* pode ser reproduzido em qualquer máquina com as mesmas dependências, com o uso de um container Docker.

O modelo final (`rf_opt`) atingiu F1 de 0,9980 para `Attack` no *holdout*, com *gap* treino–teste de apenas +0,0005, indicando boa generalização e ausência de overfitting relevante. Concluímos que Random Forest e Hist Gradient Boosting se sobressaíram na avaliação proposta. A detecção prática, contudo, ainda demanda endereçar classes de ataque extremamente raras, custo computacional em escala de produção, robustez adversarial [He et al. 2023] e validação fora do conjunto de treino.

Como perspectivas, propomos:

- **Classificação hierárquica:** estágio binário `BENIGN/ATTACK` seguido de multi-classe sobre fluxos alertados, mitigando o desbalanceamento de categorias como `Heartbleed` e `Web_Attack_Sql_Injection`.
- **Avaliação *cross-dataset*:** treinar no `CICIDS-2017` e testar em `CICIDS-2018` ou `UNSW-NB15`.
- **Técnicas de balanceamento:** contrastar `class_weight=balanced` com `SMOTE`, *undersampling* e abordagens híbridas, sobretudo em cenários multi-classe.

Referências

- Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79.
- He, K., Kim, D. D., and Asghar, M. R. (2023). Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 25(1):538–566.
- Jairu, P. and Mailewa, A. B. (2022). Network anomaly uncovering on `CICIDS-2017` dataset: A supervised artificial intelligence approach. In *Proceedings of the 2022 IEEE International Conference on Electro Information Technology (eIT)*, pages 606–615.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774.
- Maseer, Z. K., Yusof, R., Bahaman, N., Mostafa, S. A., and Foozy, C. F. M. (2021). Benchmarking of machine learning for anomaly based intrusion detection systems in the `CICIDS2017` dataset. *IEEE Access*, 9:124358–124372.
- Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116.
- Verizon Business (2026). 2026 data breach investigations report. Technical report, Verizon Business. Acesso em: 24 maio 2026.